

Herald



Herald — Participant Identity, Attribution & Notification-Tagging (DESIGN CONTRACT)

Field	Value
Revision	1
Created	2026-05-31
Status	active — implementation contract for the participant/attribution feature
Authority	Mandatory rules also restated in HelixConstitution Constitution.md/CLAUDE.md/AGENTS.md/QWEN.md (inherited per §11.4.35)

This is the **single authoritative contract** every implementation stream codes against. Operator mandate (2026-05-31): every messenger must relate messages to **participants (Subscribers/Users)**; workable items gain `created_by` + `assigned_to`; notifications **@-tag** the right participant per a fixed rule matrix; the same logical person may have a **different username on every messenger**.

1. Identity model (logical participant + per-channel handle)

A **Participant** (logical Subscriber/User) is one person/agent, with potentially a DIFFERENT username on every messenger. Backed by the existing PG tables:

- `subscribers` (the logical party): `handle` (canonical, messenger-neutral), `display_name`, `kind` ∈ {human, agent, service}.
- `subscriber_aliases` (per-channel handle): `subscriber_id`, `channel`, `channel_user_id`, + **NEW** `username TEXT` (the per-channel @handle used for tagging — distinct from `channel_user_id` which is the chat/user id). `UNIQUE (channel, channel_user_id)`.

Canonical handle = the string stored in items' `created_by` / `assigned_to`. Closed set:

- Claude — the system agent (reserved sentinel; `kind=agent`). NEVER tagged.
- a human's **canonical handle** — defaults to their Telegram @username (Telegram is the primary messenger) but is messenger-neutral; per-channel @usernames are resolved via `subscriber_aliases`.

Operator = the one human who drives the system via the Claude Code CLI. Designated by env var, NOT a DB flag:

- HERALD_TGRAM_OPERATOR_USERNAME (e.g. @milos85vasic) — per-messenger; generalizes to HERALD_<CHANNEL>_OPERATOR_USERNAME (HERALD_SLACK_OPERATOR_USERNAME, ...).
- The operator's **canonical handle** = their Telegram operator username (e.g. @milos85vasic).
- The operator is a normal Participant whose handle equals the operator env value.

commons types (the contract surface):

```
const SystemAgentHandle = "Claude" // reserved created_by/  
    assigned_to sentinel; never tagged  
  
type Participant struct {  
    Handle      string           // canonical handle (e.g.  
        "@milos85vasic" or "Claude")  
    DisplayName string  
    Kind        string           // "human" | "agent" |  
        "service"  
    Usernames   map[string]string // channel -> "@username" on  
        that channel  
}  
  
// IdentityResolver bridges the per-channel runtime world to  
// canonical handles.  
type IdentityResolver interface {  
    // Inbound: map a received message's sender to a canonical  
    // handle.  
    ResolveSender(channel, channelUserID, username string)  
        (handle string)  
    // Outbound: the @username for a canonical handle on a target  
    // channel (ok=false if the  
    // participant has no alias on that channel – cannot tag  
    // someone not on that messenger).  
    UsernameFor(handle, channel string) (username string, ok  
        bool)  
    // OperatorHandle returns the canonical operator handle (from  
    // HERALD_<CH>_OPERATOR_USERNAME).  
    OperatorHandle() string  
}
```

2. Attribution rules — who sets created_by / assigned_to

created_by (who opened/assigned the item):

- Opened via the **Claude Code CLI prompt** (operator-driven) → created_by = OperatorHandle().

- Opened by **System/Claude** detecting an issue/task/improvement/missing-feature → `created_by = "Claude"`.
- Received **through Herald** (a subscriber message) → `created_by` = the sender's canonical handle (resolved via `ResolveSender` from the message's `@username` + other data).

`assigned_to`:

- **Default** = `OperatorHandle()` (the operator's canonical handle).
- May be overridden explicitly (e.g. a prompt or message that assigns to `@someoneelse`).

Both columns store the **canonical handle string** (self-contained in the SSoT/MD).

3. Notification-tagging matrix — who gets @-mentioned

On any workable-item event, the outbound notification dispatched to each messenger channel/group @-tags the participant(s) who must be aware, resolved to that channel's `@username`:

```
mentions = {}
if assigned_to is a human handle AND assigned_to != Operator: mentions +
if created_by is a human handle AND created_by != Operator AND created_b mentions +

# "Claude" is NEVER tagged (it is the system).
# Operator is NEVER tagged (the operator drives the system; no self-ping).
# de-dup; for each mention resolve UsernameFor(handle, channel) — skip if
```

This exactly satisfies the operator's stated rules:

- **assigned to Operator** → **no tag** (`assigned_to==Operator` skipped; `created_by==Operator` skipped).
- **opened by Operator, assigned to another** → **tag the assignee** (`assigned_to≠Operator` tagged).
- **opened by a non-Operator non-Claude subscriber** → **tag** (`created_by` tagged).

commons contract:

```
// MentionsFor returns the canonical handles to @-mention for an
// item event on `channel`,
// per the matrix above. Resolution to @username happens via
// IdentityResolver.UsernameFor.
func MentionsFor(createdBy, assignedTo, operatorHandle, channel
    string,
    r IdentityResolver) []string
```

4. Storage & format changes

4a. Workable-items SQLite SSoT (constitution tool — the canonical schema)

`items` table gains:

- `created_by TEXT NOT NULL DEFAULT ''`

- `assigned_to` TEXT NOT NULL DEFAULT '' Parser reads `**Created-By:** <handle>/**Assigned-To:** <handle>` from each item block; renderer writes them; byte-identical round-trip preserved. `validate` accepts empty (legacy).

4b. `commons_workable` (Herald's mirror store)

Item struct gains `CreatedBy string + AssignedTo string`. SQLite items table + the Go migration add the two columns (default ""). `ParseTracker` reads the two MD fields; the change-feed `Diff` emits `item.field.changed` for `created_by/assigned_to` changes.

4c. Markdown trackers + exports

- Herald pipe-table `Issues.md/Fixed.md`: add **Created-By** + **Assigned-To** columns.
- ATMOSphere heading-format trackers: add `**Created-By:** / **Assigned-To:**` fields.
- Regenerate HTML/PDF/DOCX siblings + summaries for every edited doc.

4d. PG subscribers/subscriber_aliases

- Migration: `ALTER TABLE subscriber_aliases ADD COLUMN username TEXT`; (the per-channel `@handle` for tagging). Backfill where known.

5. Wiring points

- **Inbound** (`pherald/internal/inbound`): when a message opens/updates an item, set `created_by` via `ResolveSender`; default `assigned_to = OperatorHandle()` unless the message assigns explicitly. When Claude opens an item, `created_by = "Claude"`.
- **Outbound** (`pherald/internal/workflow`): `RenderChange + Notifier` call `MentionsFor` and prepend/append the resolved `@usernames` to the dispatched message body (per channel).
- **tgram adapter** (`commons_messaging/channels/tgram`): render a mention as `@username` (a Telegram username mention reaches a group member). Other adapters render their channel's mention syntax (Slack `<@U...>`, etc.) — future.

6. Anti-bluff testing mandate (MANDATORY — §107 / Helix §11.4)

Every layer ships unit + integration + **E2E + full-automation** tests that produce **real captured evidence** (no metadata-only/absence-of-error PASS, no false +/-). 100% of the feature's behaviour is exercised against real components:

- real SQLite round-trip with the new columns (byte-identical);
- real `IdentityResolver` over a real `subscribers/subscriber_aliases` (PG + in-memory);
- the tagging matrix proven by a truth-table test (every cell) + a mutation that flips one cell must FAIL;
- E2E: a real item event → a real dispatched message whose body contains exactly the expected `@usernames` (recording channel sink) — and a NEGATIVE case proving the Operator is NOT tagged.

- All evidence committed under docs/qa/<run-id>/.

7. Constitution authority (MANDATORY, inherited)

These rules are restated as mandatory constraints in HelixConstitution Constitution.md + CLAUDE.md + AGENTS.md + QWEN.md (root definitions, inherited per §11.4.35), then in Herald's + ATMOSphere's docs. The anti-bluff covenant (§107 / §11.4) is a hard precondition of every PASS here.